

RESEARCH PROPOSAL

Improving Optimizing Prompts with Strategy Selection (OPTS) for legal contexts

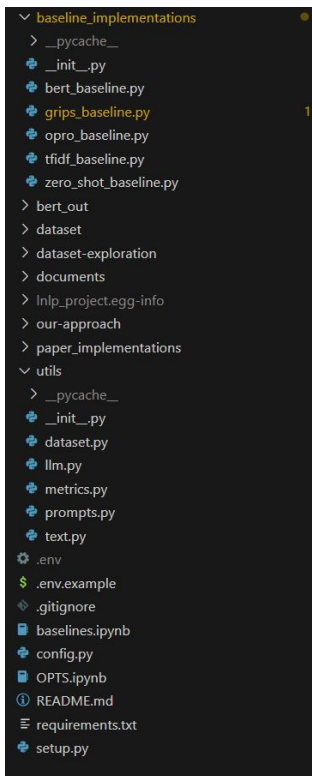
Ashizawa, Rin, et al. - arXiv:2503.01163 (2025)

Michael Bressler

Master Practical Course - Legal Natural Language Processing Lab, 11.06.2025

Short recap of **our baseline classifications**

Refactor of Codebase for efficient workflow



...

Binary classification									
	Baseline	Parameters	Sample N	accuracy	micro_f1	macro_f1	precision	recall	support
0	TF-IDF Logistic	model=logistic max_features=10000	20	0.95	0.95	0.487179	1.0	0.95	20
1	TF-IDF SVM	model=svm max_features=10000	20	0.85	0.85	0.459459	1.0	0.85	20
2	Zero-Shot LLM	model=openrouter sample_size=20	20	1.00	1.00	1.000000	1.0	1.00	20
3	OPRO	iterations=3	20	0.50	0.50	0.333333	1.0	0.50	20

...

Multilabel classification										
	Baseline	Parameters	Sample N	micro_f1	macro_f1	micro_precision	micro_recall	macro_precision	macro_recall	support
0	TF-IDF Logistic	model=logistic max_features=10000	20	0.0	0.0	0.0	0.0	0.0	0.0	20
1	TF-IDF SVM	model=svm max_features=10000	20	0.0	0.0	0.0	0.0	0.0	0.0	20
2	Zero-Shot LLM	model=openrouter sample_size=20	20	0.0	0.0	0.0	0.0	0.0	0.0	20

The **Problem**

Problem

DISCUSSION

P

Consumers depend on expensive lawyers to identify unfair clauses in consumer contracts

P

LLMs struggle to perform classification tasks in complicated legal contexts

P

State-Of-The-Art automated means to analyze contracts are not capable of fully satisfying legal needs of consumers. Research on Automatic Prompt Optimization has potential to yield improvements.

Research **Question**

Research Questions

TO BE ANSWERED

1)

Can OPTS be improved by adding additional strategies designed for legal reasoning problems?

2)

Can OPTS be improved by supplying relevant legal contextual information to the prompt in the prompt optimization stage and at inference time?

Optimizing Prompts with Strategy Selection

OPTS

Optimizing Prompts with Strategy Selection

METHOD FROM PAPER

Ashizawa, R., Hirose, Y., Yoshinari, N., Uchida, K., & Shirakawa, S. (2025). **Bandit-Based Prompt Design Strategy Selection Improves Prompt Optimizers.** arXiv preprint arXiv:2503.01163.

arXiv:2503.01163v1 [cs.AI] 3 Mar 2025

Bandit-Based Prompt Design Strategy Selection Improves Prompt Optimizers

Rin Ashizawa Yoichi Hirose Nozomu Yoshinari
Kento Uchida Shinichi Shirakawa

Yokohama National University

{ashizawa-rin-gs,hirose-youichi-kc,yoshinari-nozomu-ry}@ynu.jp
{uchida-kento-fz,shirakawa-shinichi-bg}@ynu.ac.jp

Abstract

Prompt optimization aims to search for effective prompts that enhance the performance of large language models (LLMs). Although existing prompt optimization methods have discovered effective prompts, they often differ from sophisticated prompts carefully designed by human experts. Prompt design strategies, representing best practices for improving prompt performance, can be key to improving prompt optimization. Recently, a method termed the Autonomous Prompt Engineering Toolbox (APET) has incorporated various prompt design strategies into the prompt optimization process. In APET, the LLM is needed to implicitly select and apply the appropriate strategies because prompt design strategies can have negative effects. This implicit selection may be suboptimal due to the limited optimization capabilities of LLMs. This paper introduces Optimizing Prompts with Strategy Selection (OPTS), which implements explicit selection mechanisms for prompt design. We propose three mechanisms, including a Thompson sampling-based approach, and integrate them into EvoPrompt, a well-known prompt optimizer. Experiments optimizing prompts for two LLMs, Llama-3-8B-Instruct and GPT-4o mini, were conducted using BIG-Bench Hard. Our results show that the selection of prompt design strategies improves the performance of EvoPrompt, and the Thompson sampling-based mechanism achieves the best overall results. Our experimental code is provided at <https://github.com/shirakawa/OPTS>.

2024). Since well-crafted prompts improve the performance of LLMs, prompt engineering (i.e., designing better prompts) plays a key role in the area (Bsharat et al., 2024; Schulhoff et al., 2024). Despite its importance, prompt engineering is laborious as it requires a lot of time for refinement and sufficient knowledge of the tasks. To design effective prompts with less effort, research on prompt optimization has been actively conducted. In particular, discrete prompt optimization, which optimizes prompts within the natural language space, has attracted attention. This approach is valuable as it typically allows for the optimization of prompts for black-box LLMs, such as GPT-4, while also providing interpretable results (Chang et al., 2024). To explore effective prompts within the large natural language space, several methods have been proposed, which include emulating evolutionary algorithms using LLMs (Guo et al., 2024; Fernando et al., 2023; Cui et al., 2024). These methods have discovered effective prompts, but they often differ from sophisticated prompts carefully designed by human experts.

Prompt design strategies, which provide guidelines for creating effective prompts, can be key to boosting the performance of prompt optimizers. In fact, Chain-of-Thought (CoT; Wei et al., 2022) and Role Prompting (Wang et al., 2024a) have been employed in prompt optimization, leading to better prompts (Agarwal et al., 2024). Recently, Koppel and Valogianni (2024) proposed a method termed the Autonomous Prompt Engineer-

Pseudocode

APET_OptimizePrompt(original_prompt, strategy_descriptions):

1. Construct Meta-Prompt:

```
// A system prompt that instructs the LLM to reformulate the original prompt
```

```
meta_prompt ← ""
```

```
Imagine yourself as an expert in the realm of prompting techniques for LLMs.  
You have the following available strategies:
```

```
{for i in 1..K: "- Strategy {i}: {strategy_descriptions[i]}"}"
```

```
Your approach is methodical and analytical, yet creative. You will implicitly  
choose which strategies to apply and combine them to optimally reformulate  
the following user prompt:
```

```
"{original_prompt}"
```

```
Return only the fully reformulated prompt as your output.
```

```
""
```

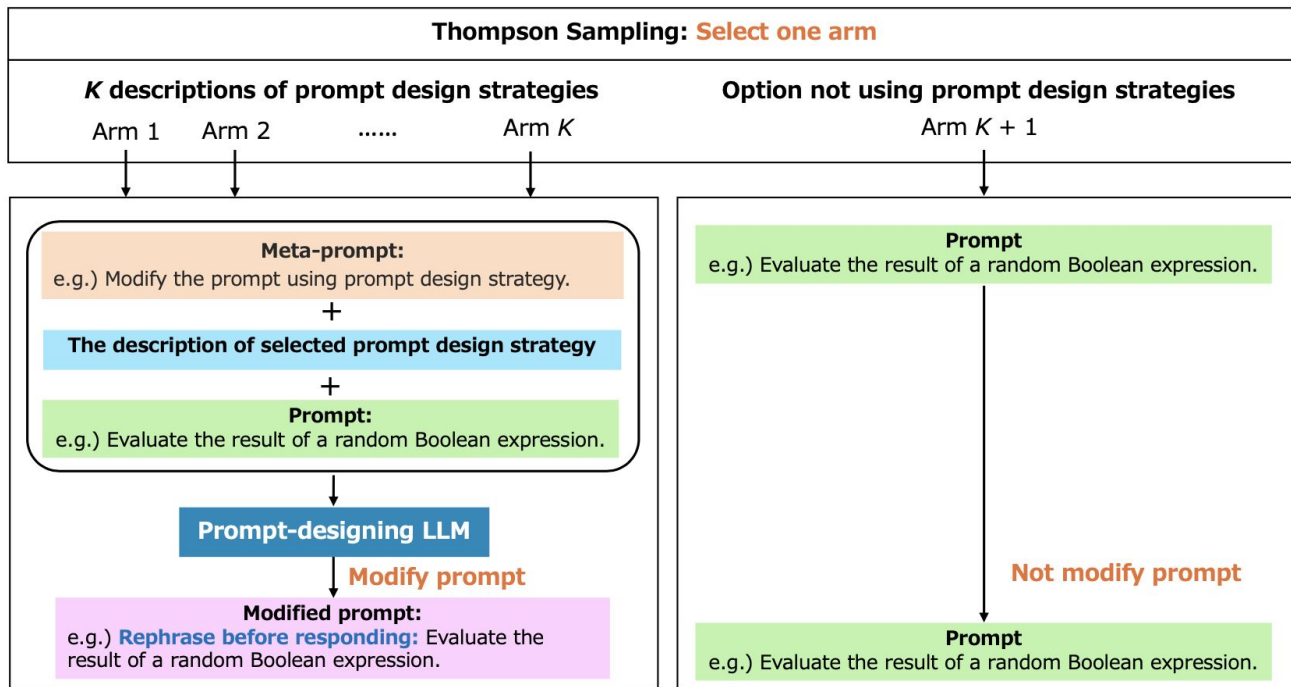
2. Call LLM once:

```
improved_prompt ← LLM.generate(meta_prompt)
```

3. Return improved_prompt

Based on: Ashizawa, R., Hirose, Y., Yoshinari, N., Uchida, K., & Shirakawa, S. (2025). Bandit-Based Prompt Design Strategy Selection Improves Prompt Optimizers. *arXiv preprint arXiv:2503.01163*.

OPTS



Source: Ashizawa, R., Hirose, Y., Yoshinari, N., Uchida, K., & Shirakawa, S. (2025). Bandit-Based Prompt Design Strategy Selection Improves Prompt Optimizers. *arXiv preprint arXiv:2503.01163*.

OPTS Curated Strategies

ExpertPrompting	Crafting an expert who is an expert at the given task, by writing a high-quality description about the most capable and suitable agent to answer the instruction in second person perspective.
Chain-of-Thought	Explaining step-by-step how the problem should be tackled, and making sure the model explains step-by-step how it came to the answer. You can do this by adding "Let's think step-by-step".
Tree-of-Thought	Imagining three different experts who are discussing the problem at hand. All experts will write down 1 step of their thinking, then share it with the group. Then all experts will go on to the next step, etc. If any expert realises they're wrong at any point then they leave.
Adding necessary information	Making sure all information needed is in the prompt, adding where necessary but making sure the question remains having the same objective.
Re-Reading	For a given prompt, add a phrase such as "Read the question again" that instructs the Large Language Models to reread the question before generating an answer. This strategy is particularly effective for complex tasks and helps enhance the quality and reliability of the model's outputs.
Style Prompting	Clearly define the desired style in the given prompt. For example, you might say, "Write a formal letter about..." or "Create a casual conversation discussing...". This guidance helps the model produce text that matches the requested stylistic elements, whether it's formal, informal, technical, or poetic.
Rephrase and Respond	For a given prompt, add a phrase that instructs the Large Language Models to rephrase the question before responding, such as "Rephrase and expand the question, and respond.
Making prompt specific	Make the description of the given prompt more specific. This makes it easier for Large Language Models to correctly execute prompt instructions.
Avoiding bias	To allow Large Language Models to make logical and unbiased inferences, add phrases to a given prompt that instruct it to remove opinionated content. This helps the model concentrate on providing responses based on careful analysis and logical reasoning, minimizing biases.
Shortening the prompt	If a given prompt has long instructions, make it shorter by condensing it to only the essential parts.
Emotion Prompting	At the end of the prompt, add a phrase that evokes a strong emotion. When doing so, keep the following four points in mind: 1. Define emotional goals: Identify the emotional response you want to evoke, such as encouragement, motivation, or reassurance. 2. Use positive language: Incorporate words and phrases that are positive and supportive. Examples include "believe in your abilities," "excellent," "success," and "outstanding achievements".

Proposed **Improvements**

Additional strategies proposed by us

ToS Context	Add a function {...} to the prompt to be provided at inference time to look up the n preceding and n following sentences in the ToS document using the sentence ID. Append the prompt with this context in a helpful way. (insertion method subject of prompt optimization)
Statutory Context	For a given prompt, identify one of these {...} possibly relevant legal provisions. Append these legal provisions as context to the prompt. (insertion method subject of prompt optimization)
Legal Reasoning	Edit the prompt to guide the LLM through classic legal reasoning. 1) State the goal of determining the unfairness of a clause. 2) Give a detailed definition of what could be considered an unfair clause. 3) compare the given sentence with the definition to estimate which parts of the sentence falls under that definition. 4) make a final determination based on the comparison. (phrasing of instruction subject of prompt optimization)

Implementation Stage

FURTHER STEPS

```
import random
import numpy as np
from utils.dataset import claudetteDataset
from utils.llm import get_llm_task_response
from utils.prompts import BINARY_TASK_INSTRUCTION
from utils.metrics import compute_binary_metrics, display_metrics

STRATEGIES = {
    "cot": "Let's think step-by-step before answering.",
    "emotion": "Consider the emotional tone of the sentence.",
    "rephrase": "Rephrase the sentence before judging."
}

META_PROMPT_TEMPLATE = """{strategy}
Classify whether the following clause is UNFAIR or FAIR: \"{sentence}\""""

cd = claudetteDataset()
train_df = cd.get_dataset('train')
DATA = list(zip(train_df['text'], train_df['label']))

def llm_classify(clause: str) -> str:
    resp = get_llm_task_response(user_prompt=clause, system_prompt=BINARY_TASK_INSTRUCTION)
    return resp.content.strip().upper()

def evaluate_prompt(prompt: str, label: int) -> int:
    pred = llm_classify(prompt)
    pred_label = 1 if 'UNFAIR' in pred else 0
    return int(pred_label == label)

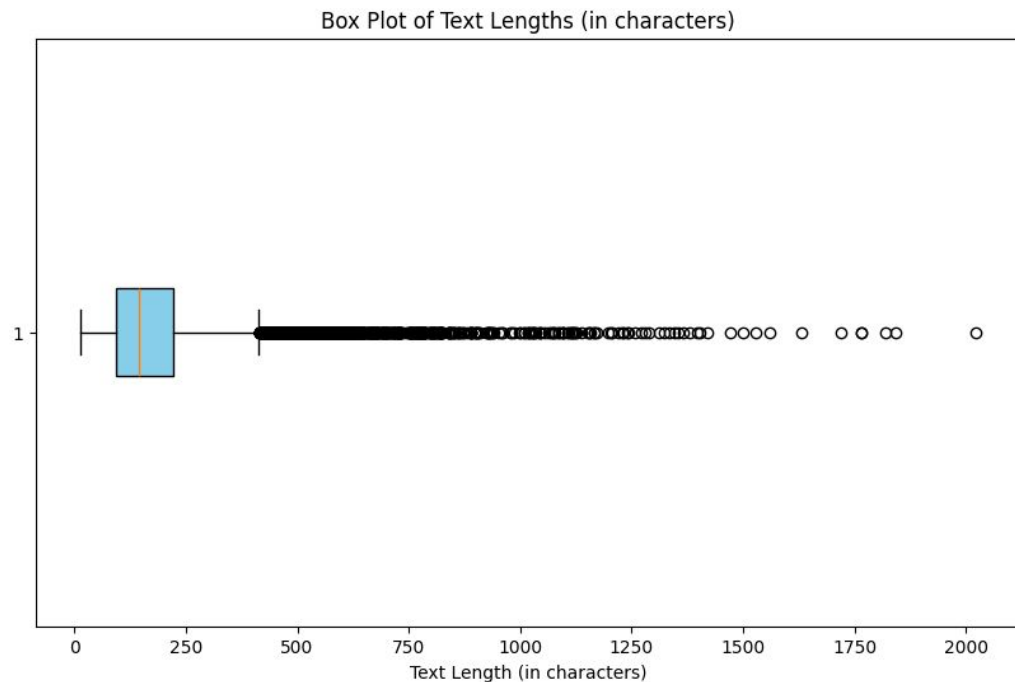
class ThompsonSampler:
    def __init__(self, strategy_names):
        self.strategies = strategy_names + ['none']
        self.alpha = {s: 1 for s in self.strategies}
        self.beta = {s: 1 for s in self.strategies}

    def sample(self):
        samples = [s: np.random.beta(self.alpha[s], self.beta[s]) for s in self.strategies]
        return max(samples, key=samples.get)

    def update(self, strategy, reward):
        if reward:
            self.alpha[strategy] += 1
        else:
            self.beta[strategy] += 1
```

Thank you
for your attention!

Distribution of Text Lengths



Descriptive Statistics for Text Lengths:

count	20417.000000
mean	179.817505
std	139.720220
min	13.000000
25%	94.000000
50%	145.000000
75%	222.000000
max	2023.000000

CLAUDETTE

an Automated Detector of Potentially Unfair Clauses in Online Terms of Service

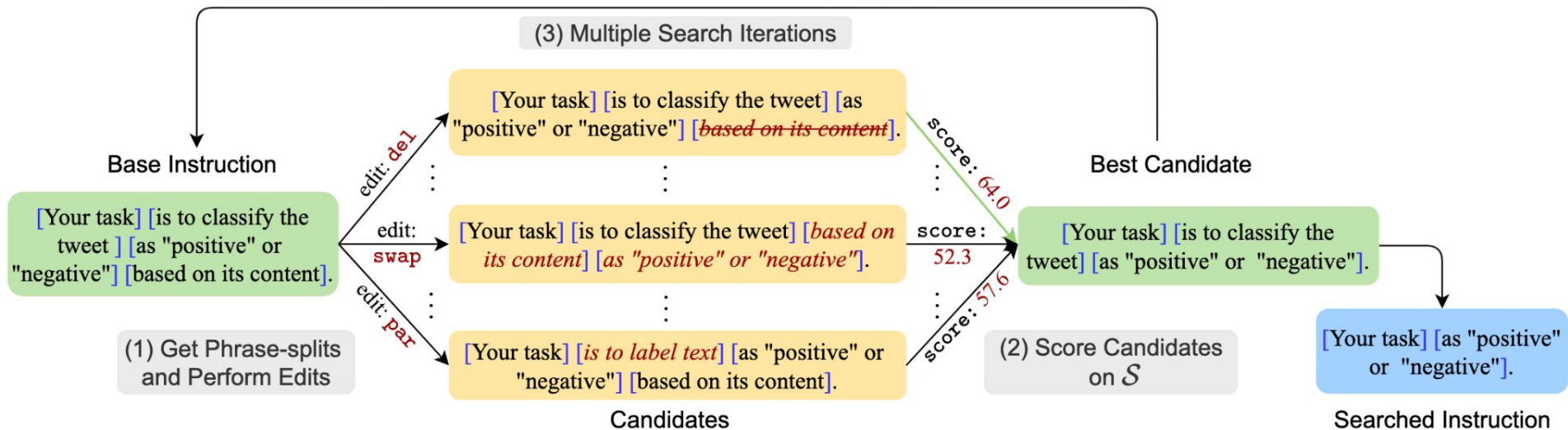
Marco, Lippi, et al. - arXiv:1805.01217 [cs.AI]

Michael Bressler

Master Practical Course - Legal Natural Language Processing Lab, 11.06.2025

GrIPS

GRIPS: Gradient-free Instructional Prompt Search



Operations

- Splitting into phrases (SuPar - just like in the paper) - Zhang et al. (2020) — "Fast and Accurate Neural CRF Constituency Parsing" / Python Library using BiLSTM or BERT (not sure)
- Delete, Add (from list of deleted), Paraphrase(Llama-3.3-70b) and Swap

Initial Prompt (from now on only Binary Classification)

Generation: via NotebookLM - Claudette paper as context

You are an expert system designed to analyze clauses found in online terms of service documents, focusing on identifying terms that may be significantly disadvantageous to the consumer.

Your task is to perform a **binary classification** on the provided text, determining if it contains a clause that should be classified as **UNFAIR** or **FAIR**.

Input: You will receive a single sentence or a short text snippet extracted from an online terms of service document.

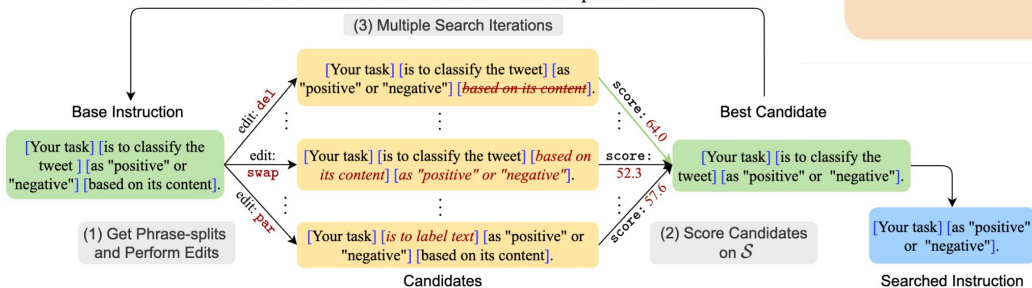
Output: You must provide **ONLY** one of the following two labels based on your analysis:

- * **UNFAIR**: If the input text contains a clause that falls under the criteria for being potentially or clearly disadvantageous to the consumer.
- * **FAIR**: If the input text contains a clause considered acceptable or beneficial to the consumer according to consumer protection principles, or if it does not contain a clause that meets the criteria for the UNFAIR category.

Your output must contain nothing other than the single word label FAIR or UNFAIR.

GrIPS

GRIPS: Gradient-free Instructional Prompt Search



I Task Instruction
Instruction: Your task is to classify the tweet as "positive" or "negative" based on its content.

E In-context Examples
Input: Glad you enjoyed it.
Output: positive
Input: Awful! My gift card ran out.
Output: negative

T Test Instance
Input: I am certainly not feeling good today...I have a cold.
Output:

Instruction-Only Prompts:
I $\oplus$ **T**
Instruction + Examples Prompts:
I $\oplus$ **E** $\oplus$ **T**
Examples-Only Prompts:
E $\oplus$ **T**

- **Model:**
- **# Iteration:**
- **# Generated candidate per iteration:**
- **Eval Size:**
- **# In-context Examples:**
 - 9 fair (random)
 - 1 from each unfair (random)

Llama-3.3-70b
 5 iteration
 2 candidates
 20 prompt
 18 in-context examples

Overview

Initial (0/0): accuracy: 0.7, f1: 0.531

Iteration 1:

- (1/1) swap: accuracy: 0.75, f1: 0.567
- (1/2) delete: accuracy: 0.75, f1: 0.567

Iteration 2:

- (2/1) paraphrase: accuracy: 0.8, f1: 0.608
- (2/2) paraphrase: accuracy: 0.8, f1: 0.608

Iteration 3:

- (3/1) add: accuracy: 0.75, f1: 0.567
- (3/2) delete: accuracy: 0.75, f1: 0.567

Iteration 4:

- (4/1) add: accuracy: 0.75, f1: 0.567
- (4/2) add: accuracy: 0.8, f1: 0.608

Iteration 5:

- (5/1) swap: accuracy: 0.7, f1: 0.531
- (5/2) paraphrase: accuracy: 0.75, f1: 0.567

The paraphrase operations in iteration 2 achieved the highest performance (0.8 accuracy, 0.608 f1), while swap operations generally showed lower performance.

INITIAL PROMPT

You are an expert system designed to analyze clauses found in online terms of service documents, focusing on identifying terms that may be significantly disadvantageous to the consumer. Your task is to perform a ****binary classification**** on the provided text, determining if it contains a clause that should be classified as ****UNFAIR**** or ****FAIR****.

****Input:**** You will receive a single sentence or a short text snippet extracted from an online terms of service document.

****Output:**** You must provide ****ONLY**** one of the following two labels based on your analysis:

* ****UNFAIR****: If the input text contains a clause that falls under the criteria for being potentially or clearly disadvantageous to the consumer.

* ****FAIR****: If the input text contains a clause considered acceptable or beneficial to the consumer according to consumer protection principles, or if it does not contain a clause that meets the criteria for the UNFAIR category.

****Your output must contain nothing other than the single word label FAIR or UNFAIR.****

accuracy: 0.7, f1: 0.531

4. ITERATION 2. CANDIDATE

As a specialized expert system your role is to scrutinize clauses within online terms of service agreements to pinpoint potentially detrimental terms affecting consumers. Your primary function involves conducting a binary classification assessment on given texts to categorize them as either UNFAIR or FAIR. You will be provided with a single sentence or a brief excerpt from an online terms of service document determining if it contains a clause that should be classified as ****UNFAIR**** or ****FAIR****. The classification of UNFAIR is applied when the input text includes a clause that may be significantly disadvantageous to the consumer. Conversely, the classification of FAIR is used when the input text contains an acceptable or beneficial clause according to consumer protection principles or lacks any clauses that meet the UNFAIR criteria. Your response should solely consist of one of the two labels FAIR or UNFAIR.

accuracy: 0.8, f1: 0.608

GrIPS Scoring

$$H = \sum_{y \in \mathcal{Y}} -p_y \log(p_y) ; p_y = \frac{1}{|\mathcal{S}|} \sum_{i=1}^{|\mathcal{S}|} \mathbb{1}(\hat{y}_i = y),$$

$$\text{score} = \text{BalancedAccuracy} + \alpha H.$$

Initial Prompt

accuracy	f1_score	combined_score	entropy
0.67	0.48	0.77	0.99

After 2 Iterations / eval_size: 100 prompts (unfortunately hit api token limit)

0.71	0.53	0.80	0.95
------	------	------	------

Best($\frac{1}{2}$): 2.iter 1.candidate

```
2025-05-25 21:23:55,2,1,paraphrase (fallback),"As a specialized expert system, your role is to scrutinize clauses within  
online terms of service agreements to pinpoint potentially detrimental terms affecting consumers. Your primary function  
involves conducting a binary classification assessment on given texts to categorize them as either UNFAIR or FAIR. You  
will be provided with a single sentence or a brief excerpt from an online terms of service document. The classification  
of UNFAIR is applied when the input text includes a clause that may be significantly disadvantageous to the consumer.  
Conversely, the classification of FAIR is used when the input text contains an acceptable or beneficial clause according  
to consumer protection principles, or lacks any clauses that meet the UNFAIR criteria. Your response should solely  
consist of one of the two labels: FAIR or UNFAIR.",0.8,0.6078431372549019
```

Best (1½) 4.iter - 2.candidid - 0.8,0.6078431372549019

2025-05-25 21:28:30,4,2,add,As a specialized expert system your role is to scrutinize clauses within online terms of service agreements to pinpoint potentially detrimental terms affecting consumers Your primary function involves conducting a binary classification assessment on given texts to categorize them as either UNFAIR or FAIR You will be provided with a single sentence or a brief excerpt from an online terms of service document determining if it contains a clause that should be classified as ****UNFAIR**** or ****FAIR**** The classification of UNFAIR is applied when the input text includes a clause that may be significantly disadvantageous to the consumer Conversely the classification of FAIR is used when the input text contains an acceptable or beneficial clause according to consumer protection principles or lacks any clauses that meet the UNFAIR criteria Your response should solely consist of one of the two labels FAIR or UNFAIR,0.8,0.6078431372549019

Initial Prompt (Multi-Label Classification)

Generation: via NotebookLM - Claudette paper as context

You are an expert system designed to analyze clauses found in online terms of service documents, focusing on identifying the categories of terms that may be significantly disadvantageous to the consumer.

Your task is to perform a **multi-label classification** on the provided text, determining if it contains a clause that should be classified as **A** or **NOTA**, **CH** or **NOTCH**, **CR** or **NOTCR**, **J** or **NOTJ**, **LAW** or **NOTLAW**, **LTD** or **NOTLTD**, **PINC** or **NOTPINC**, **TER** or **NOTTER**.

Input: You will receive a single sentence or a short text snippet extracted from an online terms of service document.

Output: You must provide **ONLY** one of the following labels per category (**A**, **CH**, etc..) based on your analysis:

- * **A**: If the input text contains a clause that falls under the criteria for being Arbitrary.
 - * **CH**: If the input text contains a clause that falls under the criteria for being Unilateral change.
- and so on...

Your output must contain nothing other than the single word label for each category: Arbitrary or Not-Arbitrary, Unilateral or Not-Unilateral and so on....

OPRO

Yang, Chengrun, et al. "Large language models as optimizers."
arXiv preprint arXiv:2309.03409
(2023).



LARGE LANGUAGE MODELS AS OPTIMIZERS

Chengrun Yang* Xuezhi Wang Yifeng Lu Hanxiao Liu
Quoc V. Le Denny Zhou Xinyun Chen*

{chengrun, xuezhiw, yifenglu, hanxiaol}@google.com
{qvl, dennyzhou, xinyunchen}@google.com

Google DeepMind * Equal contribution

ABSTRACT

Optimization is ubiquitous. While derivative-based algorithms have been powerful tools for various problems, the absence of gradient imposes challenges on many real-world applications. In this work, we propose Optimization by PROMpting (OPRO), a simple and effective approach to leverage large language models (LLMs) as optimizers, where the optimization task is described in natural language. In each optimization step, the LLM generates new solutions from the prompt that contains previously generated solutions with their values, then the new solutions are evaluated and added to the prompt for the next optimization step. We first showcase OPRO on linear regression and traveling salesman problems, then move on to our main application in prompt optimization, where the goal is to find instructions that maximize the task accuracy. With a variety of LLMs, we demonstrate that the best prompts optimized by OPRO outperform human-designed prompts by up to 8% on GSM8K, and by up to 50% on Big-Bench Hard tasks. Code at <https://github.com/google-deepmind/opro>.

OPRO Binary Classification

```
def build_meta_prompt(exemplars, solutions):
    prompt = "You are optimizing a classifier prompt. Your goal is to generate a new instruction that improves clause classification accuracy.\n\n"

    prompt += "## Training Examples:\n"
    for _, row in exemplars.iterrows():
        prompt += f"Clause: {row['text']}\nLabel: {row['label']}\n\n"

    prompt += "## Previous Instructions and Accuracy:\n"
    for text, score in solutions:
        prompt += f"Instruction: [{text}]\nScore: {score}\n\n"

    prompt += "## Task:\nGenerate a new instruction that is different from the previous ones and likely to improve accuracy. Respond with the instruction in square brackets only."
    return prompt
```

```
# Sample few-shot exemplars
exemplars = train_df.sample(5, random_state=42)

# Initialize instruction history
solutions = [("Classify if the clause is fair or unfair. Respond with 0 or 1 only.", 0.5)]
```

10 prompt iterations tried on 20 examples each (randomly sampled from train-set); prompts scored by accuracy

OPRO Binary Classification

Best Instruction:

Classify if the clause states or implies explicit and potentially severe penalties for non-compliance with a service, agreement, or policy, with a focus on the specific nature of the penalties and their potential consequences. Respond with 0 or 1 only.

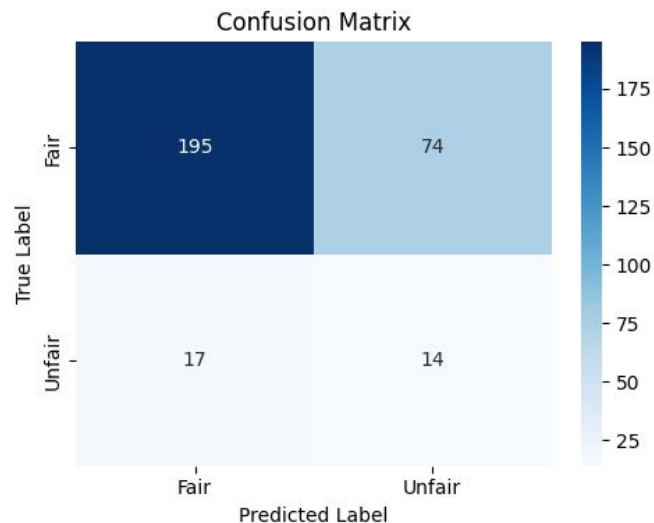
model: mistralai/mistral-7b-instruct, t=1.0

N sampled from test-set: 300

OPRO Binary Classification

	precision	recall	f1-score	support
Fair (0)	0.92	0.72	0.81	269
Unfair (1)	0.16	0.45	0.24	31
accuracy			0.70	300
macro avg	0.54	0.59	0.52	300
weighted avg	0.84	0.70	0.75	300

Accuracy : 0.6967
F1 Score : 0.2353
Precision : 0.1591
Recall : 0.4516



Thank you
for your attention!

Grips - secondary (multi-label)

You are an expert system designed to analyze clauses found in online terms of service documents, focusing on identifying the categories of terms that may be significantly disadvantageous to the consumer.

Your task is to perform a **multi-label classification** on the provided text, determining if it contains a clause that should be classified as **A** or **NOTA**, **CH** or **NOTCH**, **CR** or **NOTCR**, **J** or **NOTJ**, **LAW** or **NOTLAW**, **LTD** or **NOTLTD**, **PINC** or **NOTPINC**, **TER** or **NOTTER**.

Input: You will receive a single sentence or a short text snippet extracted from an online terms of service document.

Output: You must provide **ONLY** one of the following labels per category (**A**, **CH**, etc..) based on your analysis:

* **A**: If the input text contains a clause that falls under the criteria for being Arbitrary.

* **CH**: If the input text contains a clause that falls under the criteria for being Unilateral change.

and so on...

Your output must contain nothing other than the single word label for each category: Arbitrary or Not-Arbitrary, Unilateral or Not-Unilateral and so on....

Fine-tuned BERT as multi-label-baseline

```
filtered_train_df.head()
```

✓ 0.0s

	text	label	label_indices
2	by accessing or signing up to receive communic...	1	[9]
37	you understand and agree that mozilla reserves...	1	[3]
49	we may suspend or terminate your access to our...	1	[8]
57	except as required by law , mozilla and the in...	1	[6]
58	the collective liability of mozilla and the in...	1	[6]

```
from transformers import BertForSequenceClassification

# Load BERT with a classification head
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=9)
```

```
loss_fn = nn.CrossEntropyLoss(weight=class_weights)
```

```
# Compute Macro F1 Score
macro_f1 = f1_score(y_true, y_pred, average='macro')

print(f"Macro F1 Score: {macro_f1:.4f}")
```

```
# Compute Micro-Averaged F1 Score
micro_f1 = f1_score(y_true, y_pred, average='micro')

print(f"Micro-Averaged F1 Score: {micro_f1:.4f}")
```

TF-IDF + SVM

```
# Convert text data to TF-IDF features
vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

```
# Train an SVM classifier
svm_model = SVC(kernel='linear')
svm_model.fit(X_train_tfidf, y_train)
```

```
# Predict on test data
y_pred = svm_model.predict(X_test_tfidf)

# Evaluate model performance
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")
```

TF-IDF + SVM

Test:				
	precision	recall	f1-score	support
0	0.97	0.93	0.95	3378
1	0.55	0.73	0.63	406
accuracy			0.91	3784
macro avg	0.76	0.83	0.79	3784
weighted avg	0.92	0.91	0.91	3784
Test Micro F1: 0.9067124735729387				
Test Macro F1: 0.7861898797494948				

Table 3 Results on leave-one-document-out procedure.

Classifier	Method	P	R	F ₁
C1	SVM – Single Model	0.729	0.830	0.769
C2	SVM – Combined Model	0.806	0.779	0.784
C3	Tree Kernels	0.777	0.718	0.739
C4	Convolutional Neural Networks	0.729	0.739	0.722
C5	Long Short-Term Memory Networks	0.696	0.723	0.698
C6	SVM-HMM – Single Model	0.759	0.778	0.758
C7	SVM-HMM – Combined Model	0.848	0.720	0.772
C8	Ensemble (C1+C2+C3+C6+C7)	0.828	0.798	0.806
	Random Baseline	0.125	0.125	0.125
	Always Positive Baseline	0.123	1.000	0.217

Test:				
	precision	recall	f1-score	support
a	0.31	0.42	0.35	26
ch	0.69	0.73	0.71	78
cr	0.50	0.66	0.57	38
j	0.86	0.75	0.80	24
law	0.78	0.72	0.75	25
ltd	0.51	0.64	0.57	99
pinc	0.57	0.44	0.50	18
ter	0.71	0.60	0.65	94
use	0.71	0.79	0.75	53
micro avg	0.61	0.65	0.63	455
macro avg	0.63	0.64	0.63	455
weighted avg	0.63	0.65	0.64	455
samples avg	0.07	0.07	0.07	455
Test Micro F1: 0.6320254506892895				
Test Macro F1: 0.6273401637243258				

Table 4 Micro-averaged precision, recall and F₁ of abusive clauses for each tag category.

Tag	Precision	Recall	F ₁
Arbitration	0.832	0.814	0.823
Unilateral change	0.832	0.814	0.823
Content removal	0.713	0.780	0.745
Jurisdiction	1.000	0.941	0.970
Choice of law	0.984	0.886	0.932
Limitation of liability	0.961	0.905	0.932
Unilateral termination	0.786	0.932	0.853
Contract by using	0.949	0.957	0.953

Zero-Shot LLM

```
binary_prompt = (  
    "Given the following text, classify it as 0 (fair clause) or 1 (unfair clause):\n"  
    "Only respond with a single digit: 0 or 1.\n\n"  
    f"Text: {text}\n\nLabel (0 for fair, 1 for unfair):"  
)
```

```
multilabel_prompt = (  
    "Given the following text, which of these categories apply? "  
    "Only respond with a comma-separated list of the category codes (a, ch, cr, j, law, ltd, ter, use). "  
    "If none apply, respond with 'none'.\n\n"  
    "Categories:\n"  
    "Arbitration <a>\n"  
    "- Unilateral change <ch>\n"  
    "- Content removal <cr>\n"  
    "- Jurisdiction <j>\n"  
    "- Choice of law <law>\n"  
    "- Limitation of liability <ltd>\n"  
    "- Unilateral termination <ter>\n"  
    "- Contract by using <use>\n\n"  
    "Each code refers to a type of clause that might be unfair in a different way.\n"  
    f"Text: {text}\n\nCategories:"  
)
```

Zero-Shot LLM

Binary classification (LLM zero-shot):				
	precision	recall	f1-score	support
0	0.95	0.67	0.79	178
1	0.22	0.73	0.33	22
accuracy			0.68	200
macro avg	0.58	0.70	0.56	200
weighted avg	0.87	0.68	0.74	200
Binary Micro F1: 0.68				
Binary Macro F1: 0.5614035087719298				

t=0
n=200 from testset

Multilabel classification (LLM zero-shot):				
	precision	recall	f1-score	support
a	0.00	0.00	0.00	0
ch	0.15	1.00	0.26	5
cr	0.12	0.67	0.21	3
j	0.40	1.00	0.57	2
law	0.00	0.00	0.00	1
ltd	0.22	0.67	0.33	3
pinc	0.00	0.00	0.00	2
ter	0.16	1.00	0.28	4
use	0.09	0.67	0.16	6
micro avg	0.13	0.73	0.22	26
macro avg	0.13	0.56	0.20	26
weighted avg	0.14	0.73	0.24	26
samples avg	0.08	0.08	0.08	26
Multilabel Micro F1: 0.2222222222222222				
Multilabel Macro F1: 0.20084006065857243				

OPRO - TODO Next

- Basic Meta-Prompt Structure

Task: Optimize the prompt for text classification.

Constraints: The prompt must be concise and improve accuracy.

Previous Best Prompt: "Classify the following text into positive or negative."

Score: 85%

- Iterative Refinement

Task: Improve the prompt for summarization.

Current Prompt: "Summarize the following article in one paragraph."

Suggested Improvement: "Summarize the key points of the article in 3 sentences."

Score: 90%

- Example templates from Google DeepMind's OPRO Repository

Thank you
for your attention!

CLAUDETTE

an Automated Detector of Potentially Unfair Clauses in Online Terms of Service

Marco, Lippi, et al. - arXiv:1805.01217 [cs.AI]

Teodora Petrova, A. Erkan Turan, Michael Bressler

Master Practical Course - Legal Natural Language Processing Lab, 14.05.2025

The **Dataset**

Baseline **Classification**

Fine-tuned BERT as baseline (fair / unfair)

```
from transformers import BertForSequenceClassification

# Load BERT with a classification head (2 labels: fair and unfair)
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)
```

```
loss_fn = nn.CrossEntropyLoss(weight=class_weights)
```

```
Class weights: tensor([0.5586, 4.7683])
```

```
num_train_epochs=4,
```

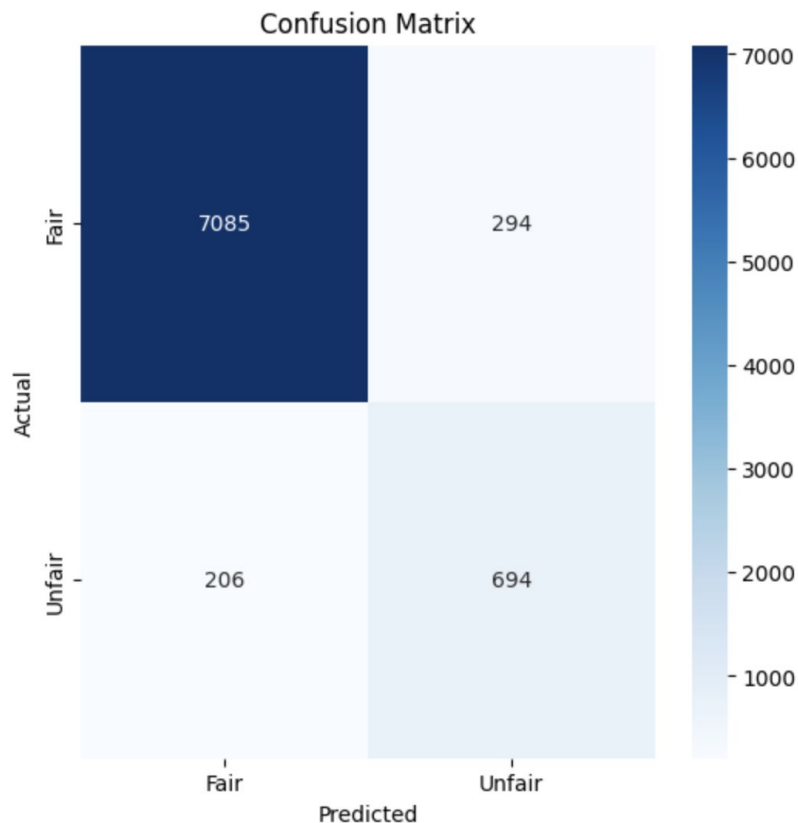
✓
54m [37] trainer.train()



[2092/2092 54:17, Epoch 4/4]

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.501900	0.343603	0.888214	0.610856
2	0.349800	0.442002	0.936839	0.703106
3	0.214900	0.496226	0.941068	0.729040
4	0.138900	0.556292	0.939482	0.721072

Fine-tuned BERT as baseline (fair / unfair)



Accuracy: 0.9396062326367919
F1 Score: 0.7351694915254238
Recall: 0.7711111111111111
Precision: 0.7024291497975709

Table 3 Results on leave-one-document-out procedure.

Classifier	Method	P	R	F ₁
C1	SVM – Single Model	0.729	0.830	0.769
C2	SVM – Combined Model	0.806	0.779	0.784
C3	Tree Kernels	0.777	0.718	0.739
C4	Convolutional Neural Networks	0.729	0.739	0.722
C5	Long Short-Term Memory Networks	0.696	0.723	0.698
C6	SVM-HMM – Single Model	0.759	0.778	0.758
C7	SVM-HMM – Combined Model	0.848	0.720	0.772
C8	Ensemble (C1+C2+C3+C6+C7)	0.828	0.798	0.806
	Random Baseline	0.125	0.125	0.125
	Always Positive Baseline	0.123	1.000	0.217

Proposed **Approach**

Pseudocode

APET_OptimizePrompt(original_prompt, strategy_descriptions):

1. Construct Meta-Prompt:

```
// A system prompt that instructs the LLM to reformulate the original prompt
```

```
meta_prompt ← ""
```

```
Imagine yourself as an expert in the realm of prompting techniques for LLMs.  
You have the following available strategies:
```

```
{for i in 1..K: "- Strategy {i}: {strategy_descriptions[i]}"}


```

```
Your approach is methodical and analytical, yet creative. You will implicitly  
choose which strategies to apply and combine them to optimally reformulate  
the following user prompt:
```

```
"{original_prompt}"


```

```
Return only the fully reformulated prompt as your output.
```

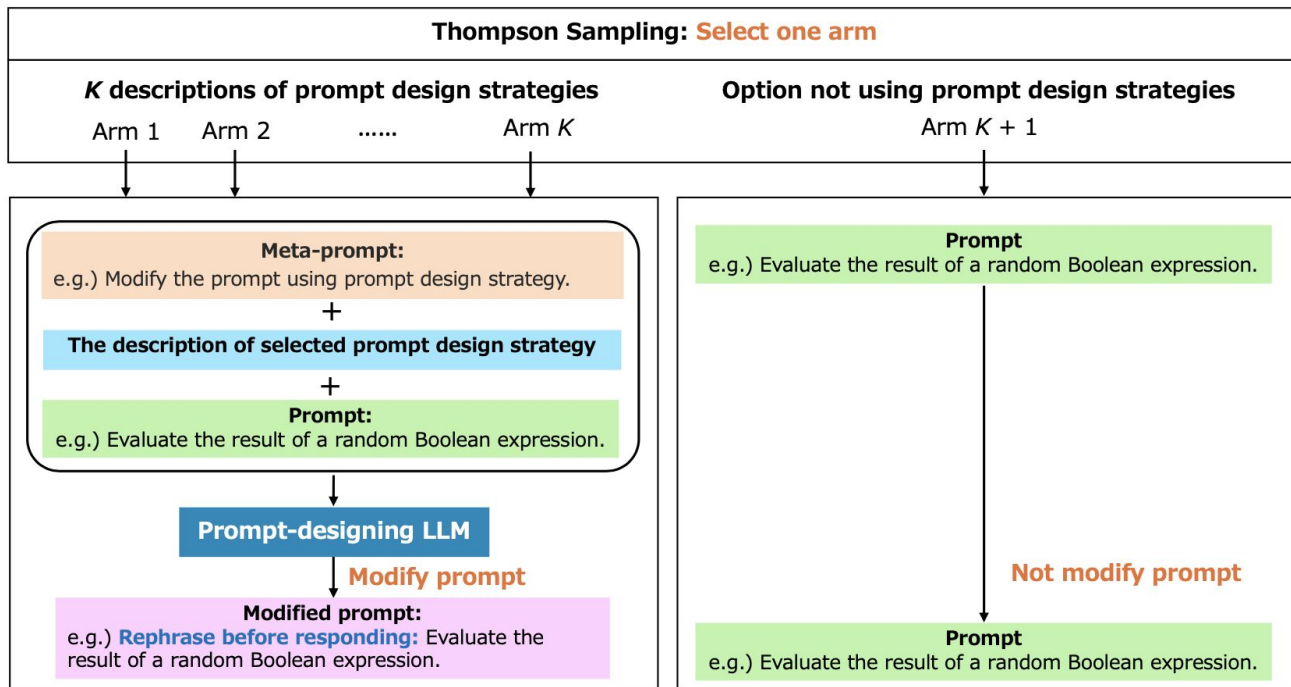
```
""
```

2. Call LLM once:

```
improved_prompt ← LLM.generate(meta_prompt)
```

3. Return improved_prompt

OPTS



Source: Ashizawa, R., Hirose, Y., Yoshinari, N., Uchida, K., & Shirakawa, S. (2025). Bandit-Based Prompt Design Strategy Selection Improves Prompt Optimizers. *arXiv preprint arXiv:2503.01163*.

OPTS Curated Strategies

ExpertPrompting	Crafting an expert who is an expert at the given task, by writing a high-quality description about the most capable and suitable agent to answer the instruction in second person perspective.
Chain-of-Thought	Explaining step-by-step how the problem should be tackled, and making sure the model explains step-by-step how it came to the answer. You can do this by adding "Let's think step-by-step".
Tree-of-Thought	Imagining three different experts who are discussing the problem at hand. All experts will write down 1 step of their thinking, then share it with the group. Then all experts will go on to the next step, etc. If any expert realises they're wrong at any point then they leave.
Adding necessary information	Making sure all information needed is in the prompt, adding where necessary but making sure the question remains having the same objective.
Re-Reading	For a given prompt, add a phrase such as "Read the question again" that instructs the Large Language Models to reread the question before generating an answer. This strategy is particularly effective for complex tasks and helps enhance the quality and reliability of the model's outputs.
Style Prompting	Clearly define the desired style in the given prompt. For example, you might say, "Write a formal letter about..." or "Create a casual conversation discussing...". This guidance helps the model produce text that matches the requested stylistic elements, whether it's formal, informal, technical, or poetic.
Rephrase and Respond	For a given prompt, add a phrase that instructs the Large Language Models to rephrase the question before responding, such as "Rephrase and expand the question, and respond.
Making prompt specific	Make the description of the given prompt more specific. This makes it easier for Large Language Models to correctly execute prompt instructions.
Avoiding bias	To allow Large Language Models to make logical and unbiased inferences, add phrases to a given prompt that instruct it to remove opinionated content. This helps the model concentrate on providing responses based on careful analysis and logical reasoning, minimizing biases.
Shortening the prompt	If a given prompt has long instructions, make it shorter by condensing it to only the essential parts.
Emotion Prompting	At the end of the prompt, add a phrase that evokes a strong emotion. When doing so, keep the following four points in mind: 1. Define emotional goals: Identify the emotional response you want to evoke, such as encouragement, motivation, or reassurance. 2. Use positive language: Incorporate words and phrases that are positive and supportive. Examples include "believe in your abilities," "excellent," "success," and "outstanding achievements".

Additional strategies proposed by us

ToS Context	Add a function {..} to the prompt to be provided at inference time to look up the n preceding and n following sentences in the ToS document using the sentence ID. Then instruct to append the prompt with this context in a helpful way.
Statutory Context	For a given prompt, identify one of these {..} possibly relevant legal provisions. Then append these legal provisions as context to the prompt.
Legal Reasoning	Edit the prompt to guide the LLM through classic legal reasoning. 1) State the goal of determining the unfairness of a clause. 2) Give a detailed definition of what could be considered an unfair clause. 3) compare the given sentence with the definition to estimate which parts of the sentence falls under that definition. 4) make a final determination based on the comparison.

Metrics

tfidf

- Unfair clauses are more uncommon than fair ones in our dataset & some categories (PINC, A) are rare-> Macro F1-Score to ensure that classes are treated equally.

- $$F1_{\text{macro}} = 1/N * \sum_{i=1}^N F1_i$$
 [1]

- modified accuracy score = BalancedAccuracy + αH [2]

[1] Opitz, J., & Burst, S. (2019). Macro f1 and macro f1. *arXiv preprint arXiv:1911.03347*.

[2] Prasad, A., Hase, P., Zhou, X., & Bansal, M. (2022). Grips: Gradient-free, edit-based instruction search for prompting large language models. *arXiv preprint arXiv:2203.07281*..

Thank you
for your attention!

The **Problem**

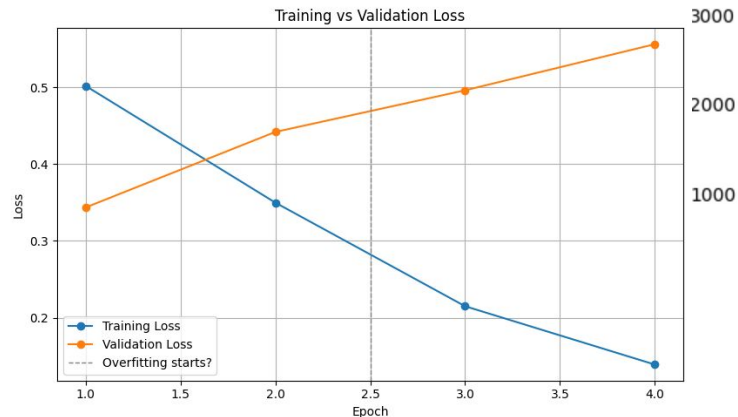
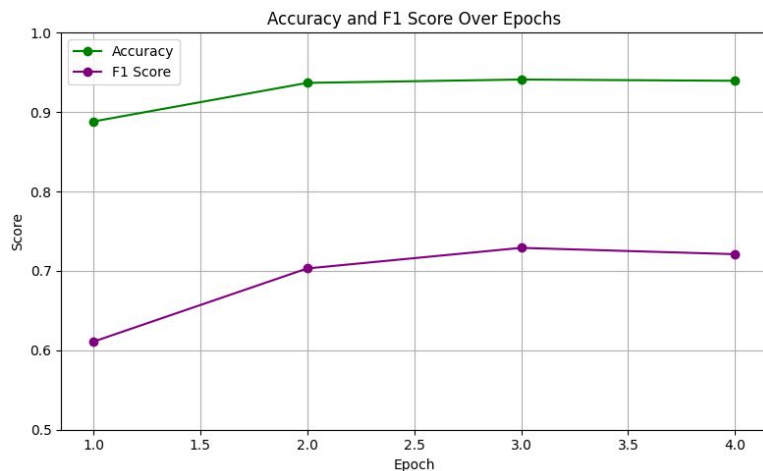
Distribution per Clause Source

Fine-tuned BERT as baseline (fair / unfair)

✓ 54m [37] trainer.train()

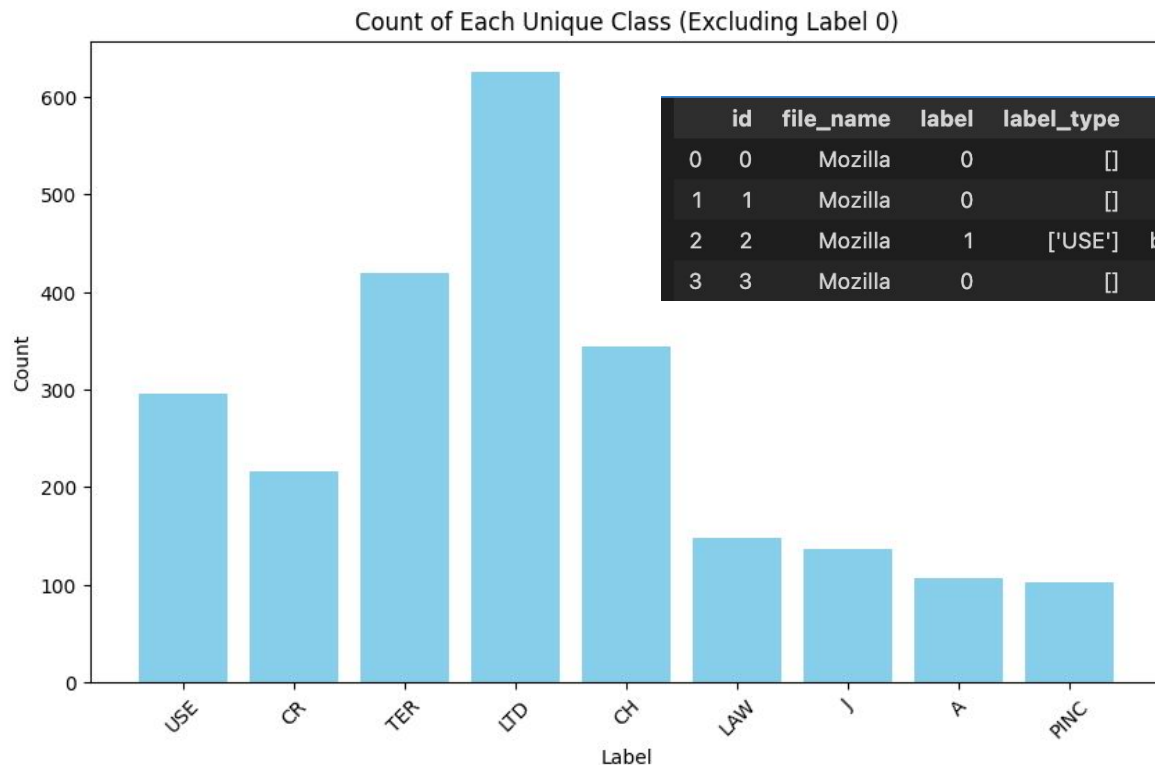
↔ [2092/2092 54:17, Epoch 4/4]

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.501900	0.343603	0.888214	0.610856
2	0.349800	0.442002	0.936839	0.703106
3	0.214900	0.496226	0.941068	0.729040
4	0.138900	0.556292	0.939482	0.721072



Recall: 0.7711111111111111
Precision: 0.7024291497975709

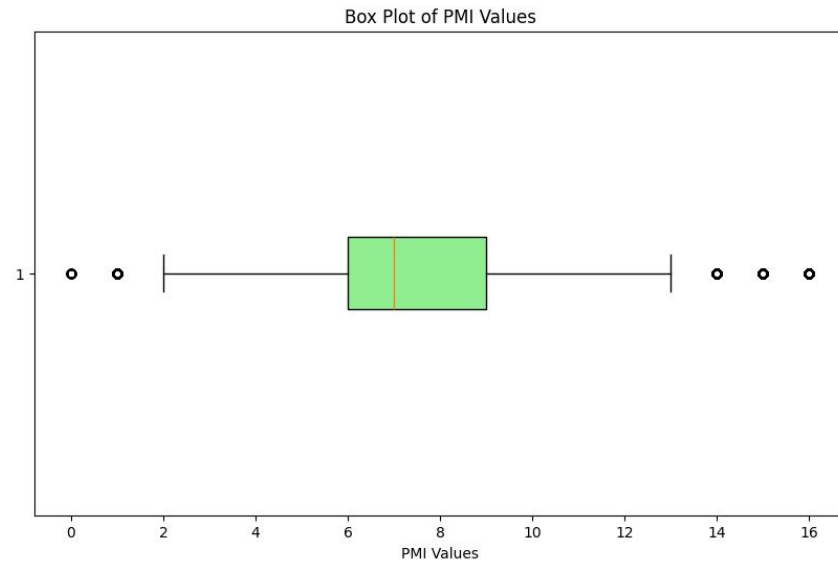
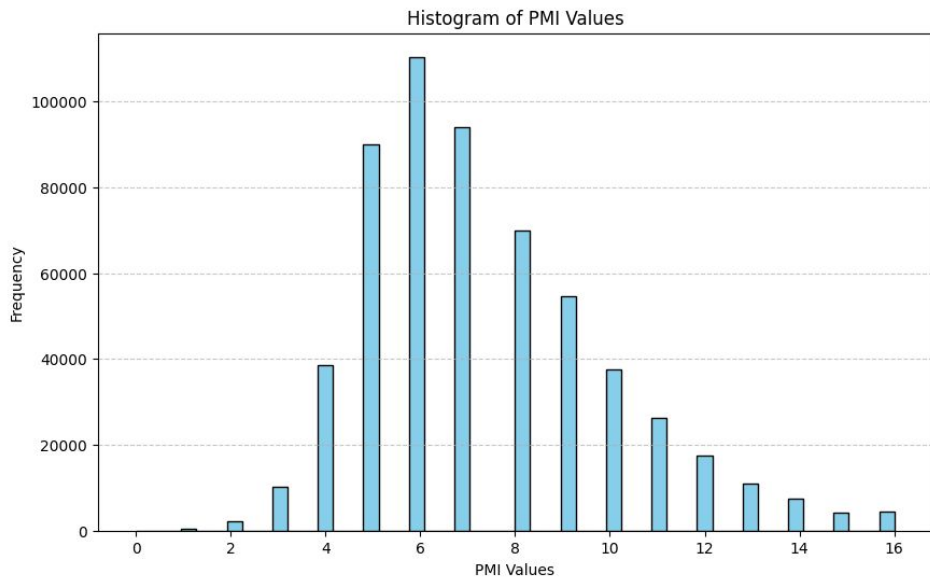
Label Distribution



id	file_name	label	label_type	text	label_indices
0	0	Mozilla	0	websites & communications terms of use	[0]
1	1	Mozilla	0	please read the terms of this entire document ...	[0]
2	2	Mozilla	1	by accessing or signing up to receive communic...	[9]
3	3	Mozilla	0	our websites include multiple domains such as ...	[0]

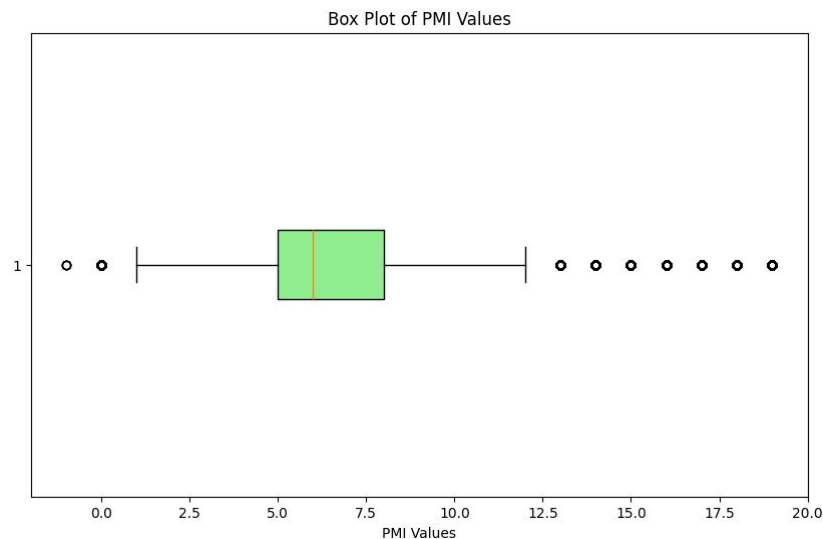
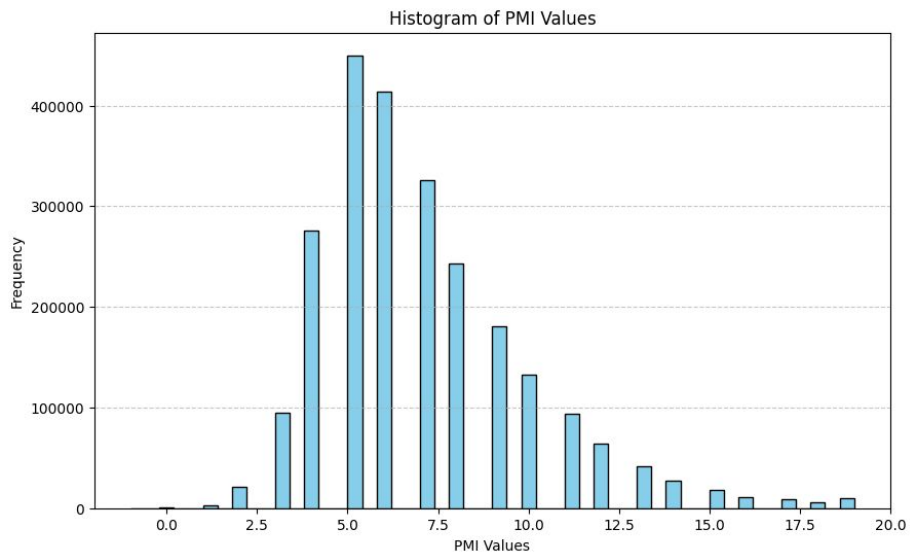
PMI (for problematic clauses)

```
LMI Statistics: {'min': np.int64(0),  
'max': np.int64(16), 'mean':  
np.float64(7.369308579960816), 'std':  
np.float64(2.574445346426766)}
```



PMI (for all clauses)

```
LMI Statistics: {'min': np.int64(-1),  
'max': np.int64(19), 'mean':  
np.float64(7.002733199288212), 'std':  
np.float64(2.877515436366146)}
```



Categories of Clause Unfairness

- A 1
- CH 2
- CR 3
- J 4
- LAW 5
- LTD 6
- PINC 7
- TER 8
- USE 9

Type of Clause	Symbol	Label Index
Arbitration	A	1
Unilateral change	CH	2
Content removal	CR	3
Jurisdiction	J	4
Choice of law	LAW	5
Limitation of liability	LTD	6
?	PINC	7
Unilateral termination	TER	8
Contract by using	USE	9

Introduction to **Instruction Tuning**

LLM Development Phases

Pre-Training

Large Text Corpora → Foundation Model

Finetuning

Instruction Tuning

RLHF

Task Specific Finetuning

Inference

Prompt Engineering

RAG

LLM Development Phases

Pre-Training

Large Text Corpora → Foundation Model

Finetuning

Instruction Tuning

RLHF

Task Specific Finetuning

Inference

Prompt Engineering

RAG

Example Instruction Pair

json

```
{  
  "instruction": "Brainstorm a list of possible New Year's Resolutions",  
  "output": "- Lose weight, - Exercise more, - Eat healthier"  
}
```

Example Instruction Pair

Foundation Model

```
{  
  "instruction": "Hi there",  
  "output": "is a casual and friendly interjection used to acknowledge  
someone's presence, initiate conversation, or express a warm welcome."  
}
```

Instruction Tuning Workflow

Pre-Training

Foundation Model

Finetuning

Large Dataset of Instruction Pairs

```
[{  
  "instruction": "Hi there!",  
  "output": "Hi! How are you doing today?"  
}, {  
  "instruction": "Hello, nice to meet you",  
  "output": "What's up! How was the weekend, Boss?"  
}, {  
  "instruction": "Brainstorm a list of possible New Year's Resolutions".
```

Instruction Tuned Model

Example Instruction Pair

Instruction Tuned Model

```
{  
  "instruction": "Hi there",  
  "output": "Hello! How are you? :) "  
}
```

LLM Use Cases

Chat

Agent

Autocomplete

...

Varying Types of Instructions wanted

Autocomplete

```
{  
  "instruction": "  
    var128763 = " michael"  
    welcome_message = "Hi there,  
  ", "output": "  
    " + str(var128763).strip().capitalize()  
  "  
}
```

Varying Types of Instructions wanted

Educational Chatbot

```
{  
  "instruction": "  
    var128763 = " michael"  
    welcome_message = "Hi there,  
  ", "output": "  
    Hi! I see you are writing python code! Great job with the syntax, let  
    me guide you through completing your script:  
  
    **Step 1**  
  "  
}
```

Varying Types of Instructions wanted

Agent

```
{  
  "instruction": "  
    var128763 = “ michael”  
    welcome_message = “Hi there,  
  ”,"output": "  
    [running database_add(var128763) ... ]  
  
    Got it! var128763 is “ michael”. I added it to the database.  
  ”  
}
```

Performance Gains

ACHIEVED THROUGH INSTRUCTION TUNING

Instruction-tuned models generally perform **significantly better** on LLM Benchmarks and Evals

Model	MT Bench
WizardLM-13b-v1.2	7.2
Vicuna-13B-16k	6.92
Mistral 7B Instruct	6.84 $\pm$ 0.065
WizardLM-13B-v1.1	6.76
Llama-2-13b-chat	6.65
Llama-2-7b-chat	6.27
Vicuna-7B-16k	6.22
Alpaca-13B	4.53

<https://mistral.ai/news/announcing-mistral-7b/>

Producing Large Instruction Datasets

FOR INSTRUCTION TUNING

Problem

Producing large datasets of instruction pairs
involves many human experts

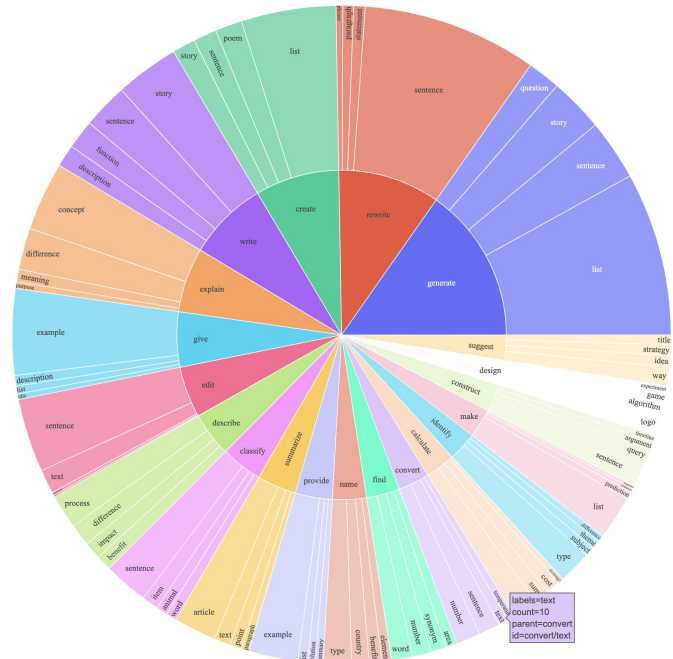
Solutions needed for industrial applications at scale.
(like in Huawei's case)

Potential for automation?

Prior Improvements to Instruction Tuning

LLM-Generated Instruction Pairs

52k instruction pairs
generated by text-davinci-003



Prompting for Instructions

tatsu-lab/stanford-alpaca/prompt.txt

You are asked to come up with a set of 20 diverse task instructions. These task instructions will be given to a GPT model and we will evaluate the GPT model for completing the instructions.

Here are the requirements:

1. Try not to repeat the verb for each instruction to maximize diversity.
2. The language used for the instruction also should be diverse. For example, you should combine questions with imperative instructions.
3. The type of instructions should be diverse. The list should include diverse types of tasks like open-ended generation, classification, editing, etc.

[..]

List of 20 tasks:

Alpaca Dataset

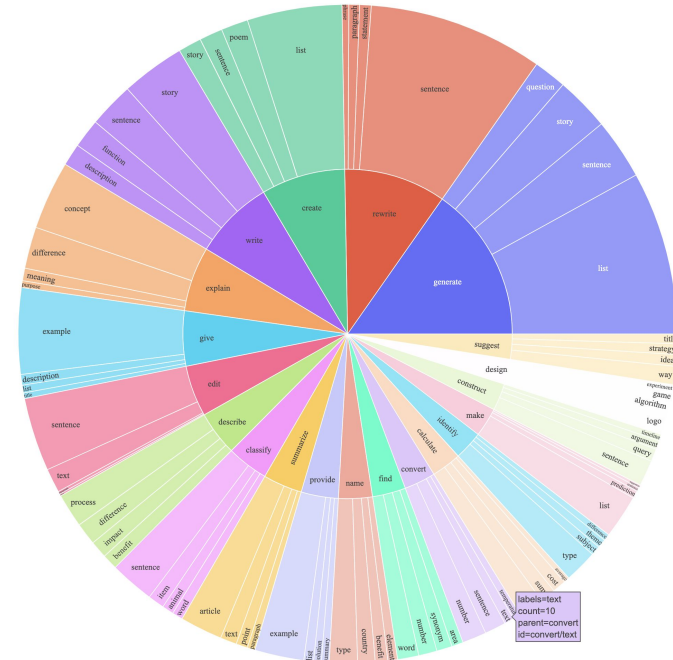
LLM-Generated Instruction Pairs

Problems

empty responses, inconsistent formats
[17-19]

Quality of instruction dataset
significantly impacts performance

rule-based revision approaches unable
to address all issues



CoachLM's Improvements to Instruction Tuning

The Dataset

PROS & CONS OF INSTRUCTION TUNING

Manual Examination by human experts
of 6k Instruction Pairs sampled from
Alpaca52k, 2.3k human revisions



Expert Revision Dataset

Resulting Expert Revision Dataset

```
{  
  "Raw Instruction": "Calculate the sum of the numbers 541, 1183, and 745.",  
  "Raw Input": "541, 1183, 745",  
  "Raw Response": "2469.",  
  "Revised Instruction": "Calculate the sum of the numbers 541, 1183, and  
745.",  
  "Revised Input": "",  
  "Revised Response": "To calculate the sum of the numbers 541, 1183, and  
745, we can add them together:  
  

$$541 + 1183 + 745 = 2469$$
  
  
Therefore, the sum of the numbers is **2469**.",  
  "Distance": 135  
},
```

Coach LM's Workflow

Foundation Model

Chat GLM

Finetuning

Expert Revision Dataset

```
[{
  "instruction": "Hi there!",
  "output": "Hi! How are you doing today?"
},{
  "instruction": "Hello, nice to meet you",
  "output": "What's up! How was the weekend, Boss?"
},{
  "instruction": "Brainstorm a list of possible New Year's Resolutions",
```

Finetuned CoachLM

Coach LM's Workflow

Model

Coach LM

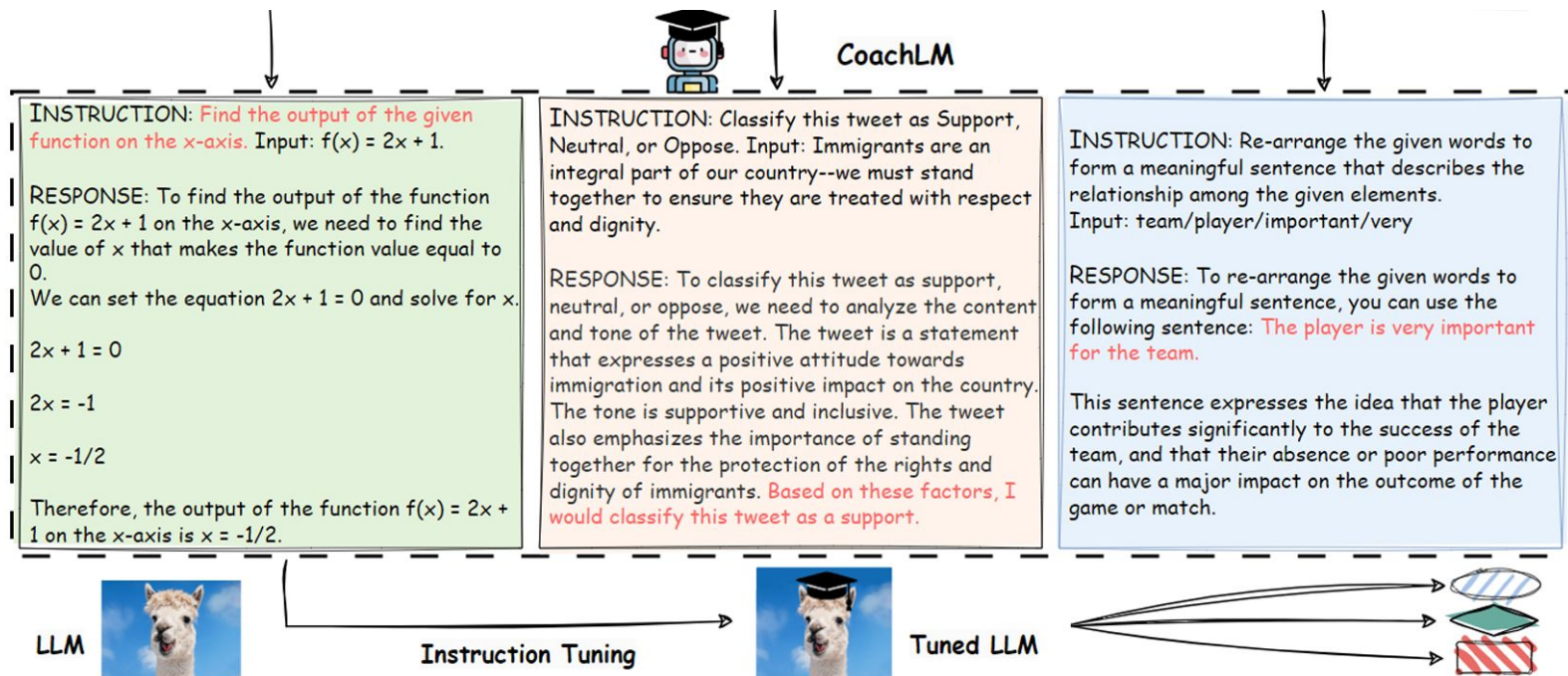
Inference

Flawed Instruction Dataset



Revised Instruction Dataset

Example Revisions



(b) The workflow of CoachLM in boosting LLM instruction tuning

Performance gained
via CoachLM-revised
instruction pairs

Quality of Instruction Pairs

ACHIEVED THROUGH LLM-REVISION

TABLE VIII
HUMAN RATINGS ON A SUBSET OF THE COACHLM-REVISED DATASET

Dataset	INSTRUCTION				RESPONSE			
	R1	R2	R3	Avg.	R1	R2	R3	Avg.
Randomly Sampled 150 Instruction Pairs □								
Original	-	-	-	-	71.1	71.2	71.3	71.2
CoachLM-revised	-	-	-	-	73.9	77.2	74.0	75.0
18 Samples in the Subset with Modified INSTRUCTIONS								
Original	76.6	74.7	77.2	76.2	67.9	70.0	68.4	68.8
CoachLM-revised	78.3	79.6	79.1	79.0	75.3	81.8	75.6	77.6

Performance gained by Instruction Tuning

VIA LLM-REVISED INSTRUCTION DATASET

TABLE IX
WIN RATES OF LLMs AGAINST REFERENCE RESPONSES ON FOUR INSTRUCTION-FOLLOWING TEST SETS RATED BY PANDALM

Model	Size	Type ^a	CoachLM150			PandaLM170			Vicuna80			Self-instruct252		
			WR1	WR2	QS	WR1	WR2	QS	WR1	WR2	QS	WR1	WR2	QS
Stronger LLMs														
LLaMA2-13b-chat [34]	13B	RL-tuned	65.3%	81.9%	91.3%	78.8%	92.2%	94.7%	54.4%	66.7%	91.3%	75.2%	92.1%	95.2%
Vicuna-13b [16]	13B	I-tuned	57.3%	66.7%	85.3%	73.8%	89.3%	93.5%	46.3%	36.4%	82.5%	67.1%	82.1%	90.5%
LLaMA2-7b-chat [34]	7B	RL-tuned	61.0%	76.2%	90.0%	78.2%	94.4%	96.5%	50.0%	50.0%	88.8%	71.0%	89.0%	94.0%
ChatGLM [32]	6B	RL-tuned	56.3%	62.7%	81.3%	76.8%	88.2%	91.8%	51.9%	60.0%	92.5%	71.4%	83.3%	89.3%
ChatGLM2 [32]	6B	RL-tuned	52.7%	55.3%	77.3%	68.8%	82.7%	90.0%	44.4%	28.6%	81.3%	64.3%	75.7%	86.5%
Alpaca-CoachLM (ours)	7B	I-tuned	67.7%	79.8%	88.0%	83.5%	95.2%	96.5%	46.9%	38.1%	83.8%	76.0%	87.4%	91.3%
Baseline LLMs														
Vicuna-7b [16]	7B	I-tuned	60.0%	71.4%	86.7%	73.5%	86.4%	91.2%	41.9%	29.0%	72.5%	68.1%	81.0%	88.9%
Alpaca [15]	7B	I-tuned	48.0%	45.7%	74.7%	62.6%	76.5%	88.8%	38.8%	20.0%	70.0%	53.8%	58.6%	81.7%
Alpaca-cleaned	7B	I-tuned	46.7%	43.1%	72.7%	62.9%	76.8%	88.8%	41.9%	21.7%	77.5%	52.8%	55.9%	79.4%
Alpaca-PandaLM [24]	7B	I-tuned	57.0%	65.7%	84.7%	72.9%	88.2%	92.9%	45.0%	31.8%	81.3%	62.7%	75.8%	88.1%
AlpaGasus [20]	7B	I-tuned	49.7%	49.2%	78.0%	65.9%	82.9%	91.8%	38.1%	17.2%	70.0%	55.6%	62.3%	82.9%
Alpaca-human (ours)	7B	I-tuned	52.0%	55.0%	82.0%	65.3%	82.5%	91.8%	42.5%	22.7%	78.8%	55.0%	62.1%	84.5%
Alpaca-CoachLM (ours)	7B	I-tuned	67.7%	79.8%	88.0%	83.5%	95.2%	96.5%	46.9%	38.1%	83.8%	76.0%	87.4%	91.3%

^a **I-tuned** is short for **Instruction-tuned**. **RL-tuned** denotes the LLMs tuned through RL pipelines in addition to instruction tuning.

Application in Production

AT HUAWEI

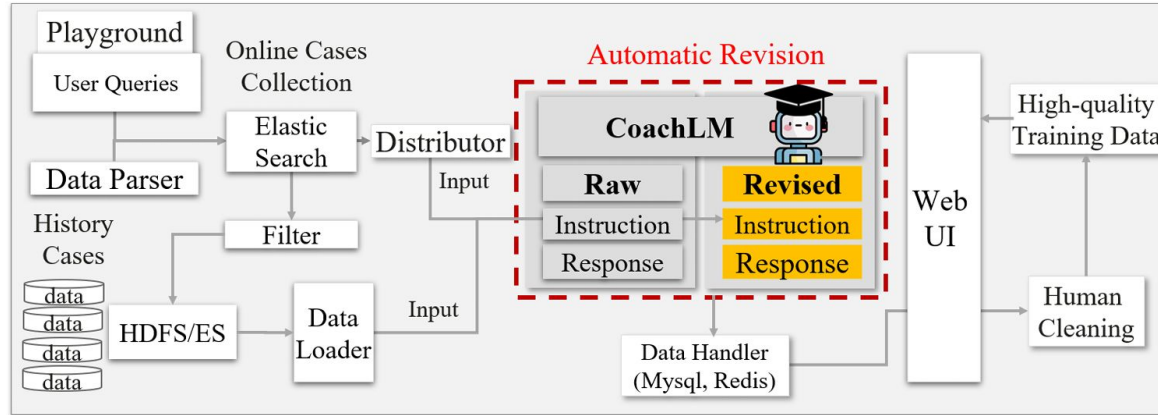


Fig. 6. Architecture of an LLM data management system at Huawei integrated with CoachLM. CoachLM automatically cleans noisy instruction pairs and mitigates human workload in data cleaning.

Pros & Cons of LLM Revised Instruction Tuning

DISCUSSION

Pros

Cost-Effectiveness

Significant Performance gains

Cons

Potential dilution of data signal

Increasing Detachment of Human involvement

Discussion

Potential Risks of Generation Augmented Instruction Tuning

Suitability for different Use Cases of Alignment:

Safety, Task Solving, Reasoning, Autocomplete, Agents

How much **human control should be given up** in Finetuning?

Are **safety evals** affected from generation-augmented instruction tuning?